

TrainTracks — Known Issues & Unresolved Items

Version: 1.0 · **Date:** March 29, 2026

Compiled from: All development sessions (Conversations #1–#9)

Issue Severity Legend

Severity	Meaning
 Critical	Feature is broken or non-functional
 Needs Polish	Functional but has rough edges or edge cases
 Integration	API or infrastructure concerns
 Cosmetic	Visual or DX issue, no functional impact

Critical — Broken / Not Working

Issue #1: GPS Crowdsourcing — CROWD- Train Still Shows as "LIVE"

Files: `useTrainStore.ts` , `useGpsCrowdsource.ts`

Status: Under active debugging

Problem: When the user broadcasts GPS via crowdsourcing, the API echoes the data back as a LIVE train (with UUID like `d78d7781-e4e`). The local CROWD- marker and the API echo coexist, causing:

- Two train markers at the same location, or
- The API echo surviving and displaying "LIVE" instead of "CROWDSOURCED"

Root Cause: The `setTrains` merge logic uses a ~1.1km (0.01°) proximity check to suppress duplicates. If the API echo position drifts beyond this radius (latency, track snapping), it survives.

Additional Concerns:

- The API may return the echo with a different line name format (`LRT-1` vs `LRT1`)
- If `crowdTrain` is `null` when `setTrains` runs (race condition), no suppression occurs

Fix Attempted: Increased suppression radius from 200m → 1.1km, split marker persistence from API reporting. Still under verification.

Issue #2: Telemetry Mapper — Status Jumps Between Stations

File: `telemetryMapper.ts`

Problem: Some live API trains show abrupt status jumps — stuck at "NOW LEAVING" then teleporting to the next station.

Root Cause: The progress-ratio calculation depends on knowing the "previous station" (direction-based). The lookup `prevStation = lineStations[targetIndex + (isSouthOrEast ? -1 : 1)]` can fail if:

- Direction strings are inconsistent (case sensitivity)
- The index goes out of bounds at terminal stations

- The API returns a position that doesn't align with expected segment geometry

Previous Fix: Replaced 300m fixed distance thresholds with progress-based ratios (12%/85%). Partially resolved.

● Needs Polish — Functional But Rough

Issue #3: Progress Bar — Visual Quirks

Files: `useStationProgress.ts` , `TripProgress.tsx`

Known Behaviors:

1. **Visual Decoupling Lag:** `progressOriginRef` / `progressTargetRef` intentionally lag behind the store to prevent premature 0% resets, but can cause brief "frozen" progress bars.
 2. **High-Water Mark Stickiness:** Since progress only goes UP (GPS drift protection), the bar can appear stuck when the train curves and Haversine distance temporarily increases.
 3. **Short Segment Zone Overlap:** For segments < 500m, the STATION (200m), LEAVING, and APPROACHING zones can overlap, causing rapid status flickers.
 4. **Post-Transfer Flash:** BUG-2/6 comment says "Do NOT reset totalProgress after transfers" — but `useTripLogic.ts` overrides it every cycle, which can cause brief 0% flash.
-

Issue #4: Transfer / NAV MODE — Edge Cases

File: `useTransferLogic.ts`

Unresolved Edge Cases:

1. **GPS Accuracy Gate Too Strict:** Exit requires `gpsAccuracy < 150m` . Inside stations, GPS accuracy is often 200m+, which can prevent NAV MODE from ever exiting even when physically at the partner station.
 2. **Transient State Flicker:** When `nextStation` briefly resolves to `null` during route recomputation, the code holds NAV MODE alive (`return` at line 181) — but this can cause a 1-frame flash of non-NAV mode UI.
 3. **Multi-Transfer Routes:** For MRT3 → LRT2 → LRT1, the first transfer works. The second transfer detection depends on `nextStation.lineId` being different from `currentStation.lineId` . If route recomputation after the first transfer is slow, the second transfer may miss its trigger.
 4. **Post-Transfer UpcomingTrainsCard:** After NAV MODE exits, status becomes `WAITING` . The card's post-transfer visibility at the new platform is untested.
 5. **Walking Distance Persistence:** `walkingDistance` in the store persists until explicitly set to `null` . The `TripProgress` checks `walkingDistance !== null` for the walking UI, which could flash stale data between transfers.
-

Issue #5: UpcomingTrainsCard — Filtering Gaps

File: `UpcomingTrainsCard.tsx`

Problem: The card filters trains by `t.status.includes('${currentStation.id}')` . For live API trains that haven't been processed by `telemetryMapper` (e.g., raw "Heading to BAC" strings), this bracket-based filter misses them entirely.

Also: After NAV MODE exits and status switches to `WAITING`, it's untested whether the card reliably appears at the new platform.

Issue #6: Station Pill Cycling & Bloom Animations

Files: `MapExplorer.tsx`, `LiveTrainLayer.tsx`

Known Issues:

- Station pill text cycling (name → congestion → docked trains) can revert to station name prematurely when CSS animation timing desyncs from the data update cycle.
 - Station bloom pulsing for dwelling trains uses Rush Hour congestion colors, but the interval timing for congestion color changes doesn't sync with the normal pulsing intervals.
 - The congestion badge in SpectatorInfoCard works correctly, but the map pill text cycling was reported as intermittently buggy.
-

● API & Infrastructure Issues

Issue #7: TrainSight API Data Quality

Files: `trainSightApi.ts`, `telemetryMapper.ts`

Concerns:

1. **Case Sensitivity:** API sometimes returns direction as `Southbound` vs `southbound`. Some string comparisons are case-sensitive.
 2. **Station Code Coverage:** `STATION_CODE_MAP` is hardcoded. New API station codes silently fall through without mapping.
 3. **MRT-7 Unmapped:** No entries in `STATION_CODE_MAP` for MRT-7 stations. All MRT-7 API data passes through unenhanced.
 4. **Status Format Variance:** The API can return `"Scheduled: Heading to NAV-MRT3"` or `"Heading to NAV"`. The regex handles both but edge cases exist.
-

Issue #8: API Token Exposure

File: `/api/trainsight/gps/route.ts`

Problem: Uses `process.env.NEXT_PUBLIC_TRAINSIGHT_TOKEN`. The `NEXT_PUBLIC_` prefix means this token is bundled into client-side JavaScript, visible in the browser's network tab.

Fix: Rename to `TRAINSIGHT_TOKEN` (without `NEXT_PUBLIC_` prefix) so it's only available server-side in the API route.

Additional Concern: No request validation — anyone can POST arbitrary GPS data through the proxy endpoint.

Issue #9: GPS Reporting — No Retry / Validation

File: `trainSightApi.ts`

Concerns:

- `reportGps()` is fire-and-forget with no retry logic
- Network failures on mobile are silently swallowed
- No validation of the API response body

- Console debugging artifacts remain in the crowdsourcing hook

Cosmetic / Developer Experience

Issue #10: SpectatorInfoCard — Upcoming Trains Section

File: `SpectatorInfoCard.tsx`

The station info "upcoming trains" section may show "Waiting for trains..." even when trains exist — because the filter requires telemetryMapper-processed status strings with bracket notation `[stationId]`.

Issue #11: Wrong Direction Alert — Terminal Station False Positives

File: `useWrongDirection.ts`

The heading comparison can fire false alerts near terminal stations where the track curves sharply (e.g., Baclaran terminus, Roosevelt terminus). The multi-confirmation gate (3 readings) mitigates this but doesn't fully prevent it.

Issue #12: Leaflet Type Definitions Missing

File: `LiveTrainLayer.tsx` (and any file importing `leaflet`)

Persistent TypeScript lint warning: `Could not find a declaration file for module 'leaflet'.` Runtime unaffected.

Fix: `npm i --save-dev @types/leaflet`

Issue #13: Dark Mode Inconsistency

File: `useTripStore.ts`

The `isDarkMode` flag exists and is consumed by `TripProgress.tsx` for shadow variations. However, `SpectatorInfoCard`, `UpcomingTrainsCard`, and several other components do not consume it—they always render in dark mode.

Issue #14: SecurityGuard DevTools Detection

File: `SecurityGuard.tsx`

The anti-tamper system uses `debugger` timing probes and nuclear DOM scrubbing. This can:

- Cause false positives during development (must be disabled manually)
 - Interfere with legitimate debugging
 - The DOM-wipe mechanism is aggressive and may affect other tabs
-

Priority Matrix

Priority	Issue	Impact	Effort
 P0	#1 CROWD- shows as LIVE	Crowdsourcing feature broken visually	Medium
 P0	#2 Telemetry status jumps	Train status unreliable	High

● P1	#4 NAV MODE GPS accuracy gate	Transfer can get stuck	Low
● P1	#4 NAV MODE multi-transfer	Second transfer may not trigger	Medium
● P1	#3 Progress bar post-transfer flash	Brief 0% visible to users	Low
● P2	#7 MRT-7 telemetry mapping	No station code entries	Medium
● P2	#8 API token in NEXT_PUBLIC_	Security exposure	Trivial
● P2	#5 UpcomingTrainsCard post-transfer	May not show at new platform	Low
● P3	#6 Station pill animation desyncs	Visual glitch on map	Medium
● P3	#12 Leaflet types missing	IDE lint warnings only	Trivial
● P3	#13 Dark mode consistency	Minor mismatched shadows	Low
● P3	#14 SecurityGuard false positives	Dev experience	Low

Recommended Fix Order

1. **#8** — Rename env var (1 line change, eliminates security risk)
2. **#12** — Install `@types/leaflet` (1 command)
3. **#1** — Debug CROWD-/LIVE echo suppression (add logging, verify line/direction match)
4. **#4a** — Relax NAV MODE GPS accuracy gate from 150m to 250m
5. **#2** — Add bounds checking to telemetryMapper prev station lookup
6. **#7** — Add MRT-7 station codes to `STATION_CODE_MAP`
7. **#3** — Add `totalProgress` HWM to prevent post-transfer flash
8. **#4c** — Test multi-transfer route (MRT3 → LRT2 → LRT1) end-to-end
9. **#5/#10** — Add raw API status fallback filter for upcoming trains
10. **#6** — Synchronize pill animation CSS timings with data update cycle

End of Known Issues Report